

Decision tree classification on imbalanced data

When the classes are imbalanced, accuracy calls a useless model excellent, so F1 is the number to report

1. What is built

A program that reads a file of card transactions, splits it into two parts, trains two classifiers and prints how well each one works. The first classifier learns nothing at all and answers not fraud to everything. The second is a decision tree.

Both are scored twice. Accuracy gives 0.9969 and 0.9994, almost the same value, so it cannot tell them apart. F1 gives 0.0000 and 0.9000, so it does. F1 is therefore the metric to report on data this imbalanced.

At the end you submit three files: `solution.py`, `execution.txt` and `report.docx`.

2. The data

`creditcard_subset.csv` is a CSV file, that is, one transaction per line and one comma between two consecutive values, with the names of the columns on the first line. It holds 32,768 card transactions made in Europe, in the order they happened. 102 of them are fraud, which is 0.311 per cent of the file. These are its first lines, with the middle columns replaced by dots so they fit on the page:

```
Time,V1,V2,V3,...,V28,Amount,Class
0.0,-1.3598071336738,-0.0727811733098,2.5363467379691,...,-0.0210530534538,149.62,0
0.0,1.1918571113148,0.2661507120596,0.1664801133532,...,0.0147241691924,2.69,0
```

Time is the seconds elapsed since the first transaction. V1 to V28 are 28 numerical features that the bank published already anonymised. Amount is the amount charged. Class is 0 for a legitimate transaction and 1 for a fraud, and it is the column to predict.

3. The two metrics

Every prediction on the test set falls into one of four boxes. The table below is the confusion matrix, and it is what the program prints as a two by two array:

	predicted not fraud	predicted fraud
really not fraud	TN, correct	FP, false alarm
really fraud	FN, fraud let through	TP, fraud caught

TN, FP, FN and TP are the four cells of the table above, and the column named cells says which of them each metric uses. The four values go from 0 to 1, and higher is better:

metric	cells	formula	what it says
accuracy	all four	$(TN + TP) / (TN + FP + FN + TP)$	share of the predictions that are correct
precision	TP, FP	$TP / (TP + FP)$	share of the alarms that were real fraud
recall	TP, FN	$TP / (TP + FN)$	share of the real frauds that were caught
F1	TP, FP, FN	$2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$	high only when precision and recall are both high

Funny: Picture a Jedi detector trained over the whole galaxy. Since Jedi are a tiny minority, a detector that answers

you are not a Jedi to everyone is correct 99.99 per cent of the time. The metric gives it top marks and the Jedi Council recruits nobody.

4. Material and setup

The archive `P1_material.zip` on the virtual campus contains `initial_solution.py`, `creditcard_subset.csv` and `README.md`. Unzip it into a folder of your own and open a terminal there.

Check the version of Python and install the two libraries:

```
$ python3 --version
Python 3.11.2
$ pip install --user pandas scikit-learn
```

`initial_solution.py` has four markers, **TODO_1** to **TODO_4**. Each step below fills one of them.

If something fails at this point, it is one of these four:

what the terminal says	what to do
python3: command not found	try python instead of python3, and use that name in every command below
pip: command not found	use python3 -m pip install --user pandas scikit-learn
ModuleNotFoundError: No module named pandas	the install went to another Python; use python3 -m pip install --user pandas scikit-learn
FileNotFoundError: creditcard_subset.csv	the terminal is in another folder; move to the folder where you unzipped the material

The numbers printed in this handout come from scikit-learn 1.4 with the seed fixed at 42. With another version the last decimal may differ. That is normal and it is not an error: report your own numbers, and check that the order between the two models is the one described here.

5. Work

The steps are done in order and each one keeps the changes of the one before. In every step, replace only the line of the marker and leave the surrounding code untouched. Nothing is undone and the file is never started again from the copy in the archive.

Step 1. Run the program as it is 10 min

From the terminal, in the folder of the practice:

```
$ python3 initial_solution.py
```

It reads the file, which takes a couple of seconds, splits it and trains the first model. It prints this:

```
transactions: 32768
frauds: 102
frauds over total: 0.311 %
training transactions: 26214
test transactions: 6554
frauds inside the test set: 20

--- model that always answers not fraud ---
accuracy: 0.9969
```

This first model answers not fraud to all 6,554 test transactions, including the 20 that are fraud. It therefore reports 20 frauds as legitimate and raises no alarm at all, so it is of no use to the bank, and yet its accuracy is 0.9969. Copy the six numbers of this block into section 3 of report . docx.

Step 2. Add the F1 score and the confusion matrix of the trivial model, **TODO_1** 20 min

Line 9 imports two functions from sklearn.metrics. Add a third one, so the line reads:

```
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score
```

Then replace the line that says **TODO_1** with these three lines:

```
print("F1: %.4f" % f1_score(y_test, trivial_answer))
print("confusion matrix:")
print(confusion_matrix(y_test, trivial_answer))
```

Run the program again. The block of the trivial model now prints:

```
--- model that always answers not fraud ---
accuracy: 0.9969
F1: 0.0000
confusion matrix:
[[6534    0]
 [   20    0]]
```

The second row of the matrix reads: of the 20 frauds in the test set, 20 were let through and 0 were caught (wrong, and this is the model whose accuracy is 0.9969). Copy this block into section 3 of report . docx.

Step 3. Train a decision tree, **TODO_2** and **TODO_3** 25 min

A decision tree splits the transactions by asking questions about their columns, one after another, until it separates the frauds. DEPTH, among the constants at the top of the file, is how many questions it may chain.

First, import it. Add this line right after the other imports, at the top of the file:

```
from sklearn.tree import DecisionTreeClassifier
```

Replace **TODO_2** with these three lines, keeping everything written in step 2. They create the tree, train it and ask it, the same three operations already used for the trivial model in step 2. Training uses x_train and y_train, and the question uses x_test. The four are variables created by the split in step 1, they are not separate files:

```
tree = DecisionTreeClassifier(max_depth=DEPTH, random_state=SEED)
tree.fit(x_train, y_train)
tree_answer = tree.predict(x_test)
```

Replace **TODO_3** with these three lines, which are the measurements of step 2 applied to tree_answer instead of trivial_answer:

```
print("accuracy: %.4f" % accuracy_score(y_test, tree_answer))
print("confusion matrix:")
print(confusion_matrix(y_test, tree_answer))
```

One line is missing, the one that prints the F1 score. Write it yourself between the other two, taking as model the line you wrote in step 2 for the trivial model and changing the name of the answer.

Run it. Training takes about half a second and prints:

```
--- decision tree of depth 5 ---
accuracy: 0.9994
F1: 0.9000
confusion matrix:
[[6532   2]
 [   2  18]]
```

18 of the 20 frauds are caught now (good), at the price of 2 legitimate transactions stopped (acceptable, a customer calls the bank). Copy this block into section 3 of report .docx.

Step 4. Print the two models together, **TODO_4** 15 min

Replace **TODO_4** with these three lines. They reuse `trivial_answer`, created in step 2, and `tree_answer`, created in step 3, both still available because they belong to the same execution:

```
print("model          accuracy   F1")
print("always not fraud  %.4f    %.4f" % (accuracy_score(y_test, trivial_answer),
f1_score(y_test, trivial_answer)))
print("decision tree    %.4f    %.4f" % (accuracy_score(y_test, tree_answer),
f1_score(y_test, tree_answer)))
```

Run it one last time. The final block prints:

```
--- comparison ---
model          accuracy   F1
always not fraud  0.9969    0.0000
decision tree    0.9994    0.9000
```

Accuracy separates the two models by 0.0025. F1 separates them by 0.9000. Copy this block into section 3 of report .docx.

Fun fact: Behind every Visa, Mastercard or American Express authorisation there is a trained classifier of the same family as the decision tree of step 3, though much larger, answering in milliseconds. Stripe, Adyen and Visa publish case studies on how they balance catching fraud against the anger of a customer whose legitimate payment is refused.

Step 5. Save the trace and write the report 30 min

Select everything the terminal printed in the last run, copy it, and paste it into a plain text file named `execution.txt`. Save your finished program as `solution.py`. Open `report_template.docx` and fill in sections 1 to 5: the first three with what you copied in the steps above, section 4 with the numbers of the trivial model, the change you made and how the numbers moved, and section 5 with 100 to 200 words on which of the two metrics you would show to the bank and why. In sections 6, 7 and 8, which are optional, delete the text between square brackets if you leave them empty. Save the file as report .docx.

Fun fact: The strongest fraud systems put a classifier and a set of rules written by domain experts side by side. Published cases from PayPal, Square and Banco Santander show the classifier never decides alone: a person validates the doubtful cases.

6. Levels

Steps 1 to 5 are the compulsory level. The two below are optional and each one is a single change to a file you already have.

Desirable extension. The depth of the tree 20 min

DEPTH is among the constants at the top of `initial_solution.py`. Change it to 3, run the program and note the three numbers. Change it to 15 and run again. With depth 3 the tree chains fewer questions and catches fewer frauds; with depth 15 it chains so many that it starts separating individual transactions instead of separating fraud. Build this table in section 6 of `report.docx`. The row for depth 5 is the one you already obtained in step 3 and is filled in here so you can check that your program is the same one; the other two rows are yours to compute:

depth	accuracy	F1	frauds caught out of 20
3	yours	yours	yours
5	0.9994	0.9000	18
15	yours	yours	yours

Under the table, state which of the three depths you would ship and why that depth is not the one with the highest accuracy.

Realistic extension. The same pipeline written by a language model 40 min

Install Ollama from ollama.com and download a model, around five gigabytes:

```
$ ollama pull qwen3:8b
$ ollama run qwen3:8b
```

Ask it for the pipeline with a request that states the file, the columns, the model and the metrics. For example:

```
Write a Python script with pandas and scikit-learn that reads creditcard_subset.csv, uses every column except Class as input and Class as target, splits it 80/20 with random_state=42 and stratify, trains a decision tree of depth 5, and prints accuracy, F1 score and the confusion matrix on the test set.
```

Save its answer as `llm_solution.py` and run it without editing it. Report in section 6 of `report.docx` whether it ran at the first attempt, whether its numbers match yours, and name every line where its script differs from yours.

If it does not run, or if it invents a column or a function that does not exist, do not repair it. Copy the error message, ask it once more adding the error message to the request, and report what happened both times. A model that fails twice is a result, not a problem, and it is worth as much as one that works.

7. Submission

One ZIP file uploaded through the virtual campus, named with your own surnames and name in lower case and without accents:

```
P1_surname1_surname2_name.zip
```

It contains exactly three files and no folders: `solution.py`, `execution.txt` and `report.docx`. A ZIP with another name, with folders inside or with a different number of files is returned without assessment. If you did any of the two extensions, say so in the corresponding section of the report and do not add files for it.

8. Reference

- Decision trees in scikit-learn: <https://scikit-learn.org/stable/modules/tree.html>
- Metrics in scikit-learn: https://scikit-learn.org/stable/modules/model_evaluation.html
- Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson and Gianluca Bontempi. Calibrating Probability with Undersampling for Unbalanced Classification. IEEE Symposium on Computational Intelligence and Data Mining, 2015.